Cruise Phase Documentation

> **Scope**: Complete documentation of the cruise phase simulation system, including steady level cruise calculations, weight-adjusted drag modeling, engine performance integration, and trajectory simulation for optimal fuel consumption analysis.

Table of Contents

1. System Overview and Objectives
2. Mathematical Foundation
3. System Architecture and Data Structures
4. Core Calculation Methods
5. Engine Performance Integration
6. Simulation Engine Implementation
7. Mission Integration and Interface
8. Validation and Quality Assurance

---

1) System Overview and Objectives

1.1 Purpose and Scope

The cruise phase simulation implements a comprehensive steady level cruise analysis system that models aircraft performance during the cruise segment of flight. The system provides accurate fuel consumption predictions, weight-adjusted drag calculations, and engine performance integration for realistic cruise trajectory simulation.

1.2 System Objectives

Primary Objectives:

- **Fuel Consumption Prediction**: Accurate modeling of fuel burn during steady level cruise

- **Performance Analysis**: Realistic assessment of cruise performance characteristics
- **Mission Integration**: Seamless connection with climb and descent phases
- **Engine Optimization**: Dynamic lever positioning for optimal fuel efficiency

Key Components:

- Steady level cruise simulation with weight-adjusted drag modeling
- Engine performance integration with dynamic lever positioning
- Atmospheric property calculations using ISA model
- Fuel consumption tracking with TSFC integration
- Integration with climb phase results for seamless mission analysis

1.3 System Flow Overview

The cruise simulation follows a logical progression:

1. **Initialization**: Extract cruise conditions from climb optimization results
2. **Atmospheric Analysis**: Calculate environmental conditions at cruise altitude
3. **Performance Calculation**: Determine thrust-drag balance and fuel consumption
4. **Trajectory Simulation**: Step through time to model complete cruise mission
5. **Results Integration**: Provide comprehensive performance data for mission analysis

---

2) Mathematical Foundation

2.1 Cruise Flight Physics

Theory: Steady level cruise flight requires thrust to exactly balance drag while maintaining constant altitude and speed. As fuel is consumed, aircraft weight decreases, affecting induced drag and requiring thrust adjustments.

Mathematical Formulation:

$$T_{required} = D_{total}$$

$$D_{total} = D_{parasitic} + D_{induced}$$

$$D_{induced} \propto C_L^2 \propto W^2$$

Where:

- `T_{required}` = Required thrust for level flight
- `D_{total}` = Total drag force
- `D_{parasitic}` = Parasitic drag (constant with weight)
- `D_{induced}` = Induced drag (varies with weight²)
- `W` = Aircraft weight

Physical Interpretation:

- **Thrust-Drag Balance**: Required for steady level flight
- **Weight Effects**: Decreasing weight reduces induced drag
- **Fuel Consumption**: Continuous weight reduction affects performance
- **Engine Response**: Dynamic lever adjustment maintains thrust balance

2.2 Weight-Adjusted Drag Model

Theory: The system models drag variation with weight changes using a simplified induced drag model that accounts for the quadratic relationship between lift coefficient and induced drag.

Mathematical Foundation:

$$C_L = \frac{W}{0.5 \times \rho \times V^2 \times S}$$

$$C_{Di} = \frac{C_L^2}{\pi \times AR \times e}$$

$$D_{induced} = 0.5 \times \rho \times V^2 \times S \times C_{Di}$$

Weight Scaling:

$$D_{induced,current} = D_{induced,base} \times \left(\frac{W_{current}}{W_{base}}\right)^2$$

2.3 Performance Parameters

Specific Excess Power (Ps):

$$P_s = \frac{(T_{total} - D) \times V_{TAS}}{W}$$

Fuel Consumption Rate:

$$\dot{m}_{fuel} = T_{total} \times TSFC$$

Thrust-Drag Equilibrium:

$$T_{required} = D_{parasitic} + D_{induced}(W)$$

---

3) System Architecture and Data Structures

3.1 Input Data Structures

3.1.1 CruiseInitialState

Purpose: Encapsulates the initial state for cruise phase, extracted from climb optimization results.

Structure:

```
@dataclass
class CruiseInitialState:
    altitude_m: float              # Cruise altitude in meters
    mach: float                    # Cruise Mach number
    weight_kg: float               # Aircraft weight after climb
    fuel_consumed_climb_kg: float  # Fuel consumed during climb
    climb_time_s: float            # Total climb time in seconds
```

Validation:

- Altitude must be within safe cruise range (1000-15000m)

- Mach must be within safe cruise range (0.3-0.9)
- Weight must be positive
- Automatic validation in `__post_init__` method

3.2 Output Data Structures

3.2.1 CruiseResults

Purpose: Complete results container for cruise simulation with trajectory arrays and summary statistics.

Trajectory Arrays:

```
time_s: np.ndarray                    # Time progression
distance_km: np.ndarray               # Distance covered
weight_kg: np.ndarray                 # Weight evolution
fuel_consumed_kg: np.ndarray          # Cumulative fuel consumption
thrust_total_N: np.ndarray            # Total thrust required
drag_N: np.ndarray                    # Total drag
fuel_flow_kgps: np.ndarray            # Instantaneous fuel flow
specific_excess_power_mps: np.ndarray # Specific excess power
lever_position: np.ndarray            # Engine lever positions
altitude_m: np.ndarray                # Altitude (constant for cruise)
mach_number: np.ndarray               # Mach number (constant for cruise)
```

Atmospheric Arrays:

```
temperature_K: np.ndarray             # Atmospheric temperature
density_kgpm3: np.ndarray             # Air density
true_airspeed_mps: np.ndarray         # True airspeed
```

Summary Statistics:

```
total_time_s: float                    # Total cruise time
total_fuel_consumed_kg: float          # Total fuel consumed
final_weight_kg: float                 # Final aircraft weight
average_fuel_flow_kgps: float          # Average fuel flow rate
average_thrust_N: float                # Average thrust required
```

3.3 System Parameters

Core Parameters:

```
# Default cruise parameters
DEFAULT_TIME_STEP_S = 60.0    # 1 minute time steps
DEFAULT_DISTANCE_KM = 1000.0  # Default cruise distance
GRAVITY_MS2 = 9.80665         # Standard gravity

# Convergence criteria
THRUST_CONVERGENCE_TOL = 1.0   # Newton tolerance for thrust balance
MAX_ITERATIONS = 50            # Maximum iterations for convergence

# Safety limits
MIN_CRUISE_MACH = 0.3         # Minimum safe cruise Mach
MAX_CRUISE_MACH = 0.9         # Maximum reasonable cruise Mach
MIN_CRUISE_ALT_M = 1000.0     # Minimum cruise altitude
MAX_CRUISE_ALT_M = 15000.0    # Maximum cruise altitude
```

---

4) Core Calculation Methods

4.1 Atmospheric Property Calculations

Function: `calculate_atmospheric_properties(altitude_m: float)`

Purpose: Calculate atmospheric properties at given altitude using ISA model.

Implementation:

```python
def calculate_atmospheric_properties(altitude_m: float) -> Tuple[float, float, float, float]:
    """Calculate atmospheric properties at given altitude using ISA model."""

    atm = Atmosphere()

    # Convert to flight level for atmosphere calculation
    flight_level = altitude_m / 0.3048
    T, p, rho = atm.calculate_atmospheric_properties(flight_level)

    # Calculate speed of sound
    a = atm.get_speed_of_sound(altitude_m)

    return T, p, rho, a
```

Returns:

- Temperature (K)
- Pressure (Pa)
- Density (kg/m³)
- Speed of sound (m/s)

4.2 True Airspeed Calculation

Function: `calculate_true_airspeed(mach: float, altitude_m: float)`

Purpose: Calculate true airspeed from Mach number and altitude.

Formula:

$$V_{TAS} = M \times a$$

Where:

- `V_{TAS}` = True airspeed
- `M` = Mach number
- `a` = Speed of sound at altitude

4.2.1 Why True Airspeed is Essential

Physical Necessity: True airspeed calculation is fundamental to cruise simulation for several critical reasons that cannot be addressed using Mach number alone:

Distance and Trajectory Calculations:

- **Distance per Step**: `distance_per_step = V_{TAS} × Δt` - Essential for determining how far the aircraft travels during each simulation time step
- **Trajectory Integration**: Required for accurate position tracking and determining when target cruise distance is reached
- **Time-Distance Relationship**: Enables conversion between time-based and distance-based mission planning

Specific Excess Power (Ps) Calculations:

$$P_s = \frac{(T_{total} - D) \times V_{TAS}}{W}$$

- **Performance Analysis**: Ps indicates aircraft's capability for climb or acceleration
- **Energy Management**: Critical for understanding available energy margins during cruise
- **Thrust-Drag Balance**: Provides quantitative measure of thrust-drag equilibrium

Aerodynamic Modeling Requirements:

- **Lift Coefficient Dependencies**: `C_L = W/(0.5 × ρ × V²_{TAS} × S)` - Induced drag calculations require actual velocity
- **Reynolds Number Effects**: True velocity affects boundary layer characteristics and aerodynamic performance
- **Dynamic Pressure**: `q = 0.5 × ρ × V²_{TAS}` - Fundamental parameter for aerodynamic force calculations

Engine Performance Integration:

- **Propulsive Efficiency**: Engine performance characteristics are typically defined in terms of true airspeed
- **Inlet Conditions**: Actual velocity through the air mass affects engine inlet dynamics
- **TSFC Dependencies**: Thrust-specific fuel consumption varies with true velocity, not just Mach number

Atmospheric Property Coupling:

- **Speed of Sound Variation**: `a = √(γ × R × T)` - Changes with altitude and temperature
- **Density Effects**: Air density variations affect both drag and engine performance
- **Temperature Dependencies**: Atmospheric temperature directly influences both speed of sound and engine efficiency

Mission Analysis Integration:

- **Phase Continuity**: Provides consistent velocity reference across different flight phases
- **Performance Comparison**: Enables comparison of cruise performance at different altitudes
- **Fuel Consumption Analysis**: True velocity affects propulsive efficiency and fuel burn rates

Implementation in Cruise Simulation:

```
# Distance calculation per time step
distance_per_step_km = (true_airspeed_mps * time_step_s) / 1000.0

# Specific excess power calculation
ps = (thrust_total_N - drag_N) * true_airspeed_mps / weight_N

# Weight-adjusted drag calculation (velocity-dependent)
induced_drag = 0.5 * rho * true_airspeed_mps**2 * S * C_Di
```

Mathematical Foundation: The relationship between Mach number and true airspeed creates the essential bridge between:

- **Constant Mach Cruise**: Maintains consistent aerodynamic characteristics
- **Variable Atmospheric Conditions**: Accounts for altitude-dependent speed of sound variations
- **Physical Performance Calculations**: Enables accurate modeling of forces, energy, and trajectory

4.3 Weight-Adjusted Drag Calculation

Function:

`calculate_weight_adjusted_drag(base_drag_N, current_weight_kg, reference_weight_kg, mach, altitude_`

Purpose: Calculate drag adjusted for current weight, accounting for induced drag variation.

Mathematical Model:

```python
def calculate_weight_adjusted_drag(base_drag_N: float, current_weight_kg: float,
                                   reference_weight_kg: float, mach: float,
                                   altitude_m: float) -> float:
    """Calculate drag adjusted for current weight (induced drag variation)."""

    # Calculate weight ratio
    weight_ratio = current_weight_kg / reference_weight_kg

    # Estimate induced drag scaling with weight²
    # Assume induced drag is ~40% of total drag for typical cruise conditions
    induced_drag_fraction = 0.4
    parasitic_drag_fraction = 1.0 - induced_drag_fraction

    # Scale drag components
    parasitic_drag = base_drag_N * parasitic_drag_fraction  # Constant
    induced_drag_base = base_drag_N * induced_drag_fraction
    induced_drag_current = induced_drag_base * (weight_ratio ** 2)

    total_drag = parasitic_drag + induced_drag_current

    return float(total_drag)
```

Key Assumptions:

- Induced drag represents 40% of total drag in cruise
- Parasitic drag remains constant with weight
- Induced drag scales with weight²

4.4 Thrust and Performance Calculations

Required Thrust Calculation:

```python
def calculate_required_thrust_cruise(drag_N: float, weight_kg: float,
                                     altitude_m: float) -> float:
    """Calculate required thrust for steady level cruise."""
    # For steady level cruise, thrust exactly balances drag
    return float(drag_N)
```

Specific Excess Power Calculation:

```python
def calculate_specific_excess_power(thrust_total_N: float, drag_N: float,
                                    weight_kg: float, true_airspeed_mps: float) -> float:
    """Calculate specific excess power for cruise."""
    weight_N = weight_kg * GRAVITY_MS2
    return (thrust_total_N - drag_N) * true_airspeed_mps / weight_N
```

Fuel Consumption Calculation:

```python
def calculate_fuel_consumption_step(thrust_total_N: float, tsfc_kg_per_N_s: float,
                                    time_step_s: float) -> float:
    """Calculate fuel consumed in one time step."""
    fuel_flow_kgps = thrust_total_N * tsfc_kg_per_N_s
    return fuel_flow_kgps * time_step_s
```

---

5) Engine Performance Integration

5.1 Dynamic Lever Positioning

Function: `find_required_lever(engine, required_thrust_N, mach, altitude_m)`

Purpose: Find the lever position required to achieve target thrust using binary search.

Algorithm:

```python
def find_required_lever(engine: EngineWrapper, required_thrust_N: float,
                        mach: float, altitude_m: float) -> float:
    """Find the lever position required to achieve target thrust."""

    required_per_engine = required_thrust_N / N_ENGINES

    # Check idle and max thrust bounds
    thrust_idle = engine.thrust_with_lever(0.0, mach, altitude_m)
    thrust_max = engine.thrust_with_lever(1.0, mach, altitude_m)

    if required_per_engine <= thrust_idle:
        return 0.0  # Idle sufficient
    elif required_per_engine >= thrust_max:
        return 1.0  # Maximum required

    # Binary search for required lever
    lever_low, lever_high = 0.0, 1.0

    for _ in range(MAX_ITERATIONS):
        lever_mid = (lever_low + lever_high) / 2.0
        thrust_mid = engine.thrust_with_lever(lever_mid, mach, altitude_m)

        if abs(thrust_mid - required_per_engine) < THRUST_CONVERGENCE_TOL / N_ENGINES:
            return lever_mid

        if thrust_mid < required_per_engine:
            lever_low = lever_mid
        else:
            lever_high = lever_mid

    return (lever_low + lever_high) / 2.0
```

Convergence Criteria:

- Maximum iterations: 50
- Thrust convergence tolerance: 1.0 N per engine
- Binary search for optimal lever position

5.1.1 Thrust Convergence Tolerance

Definition: `THRUST_CONVERGENCE_TOL = 1.0` Newton - Maximum acceptable error between required and actual thrust.

Purpose: Controls the accuracy of lever position calculations in engine performance integration.

Implementation:

- **Per-engine tolerance**: `1.0 N ÷ 2 = 0.5 N per engine`
- **Total system tolerance**: `0.5 N × 2 = 1.0 N total`
- **Precision level**: 0.002% accuracy for typical cruise conditions

Physical Meaning:

- **Thrust Balance Accuracy**: Ensures thrust-drag balance within practical limits
- **Engine Performance Modeling**: Accounts for engine performance model limitations
- **Computational Efficiency**: Balances accuracy with reasonable convergence time
- **Realistic Precision**: Matches real-world engine control system tolerances

5.2 TSFC Integration

Purpose: Integrate Thrust-Specific Fuel Consumption (TSFC) for accurate fuel flow calculations.

Implementation:

```
# Get TSFC at current engine operating point
tsfc = engine.tsfc_current()
if tsfc is None or not np.isfinite(tsfc):
    tsfc = 0.0

# Calculate fuel flow
fuel_flow_kgps = thrust_total_N * tsfc if tsfc > 0 else 0.0
```

Key Features:

- Dynamic TSFC based on current engine operating point

- Validation for invalid TSFC values
- Integration with thrust for fuel flow calculation

---

6) Code Execution Flow and Logic

6.1 System Entry Point and Initialization

Main Entry Function: `run_cruise_simulation()`

Execution Sequence:

```
# 1. System Initialization
run_cruise_simulation(
    climb_result: MinFuelSchedule,      # Input from climb optimization
    initial_mass_kg: float,             # Aircraft initial mass
    target_distance_km: float,          # Cruise distance target
    aero: AeroTables,                   # Aerodynamics model
    engine: EngineWrapper,              # Engine performance model
    time_step_s: float = 60.0,          # Simulation time step
    create_plots: bool = True           # Visualization flag
) -> CruiseResults

# 2. State Extraction from Climb Results
initial_state = extract_cruise_initial_state(climb_result, initial_mass_kg)
    ↓
    # Extract final climb conditions:
    # - final_altitude = climb_result.alt_m[-1]
    # - final_mach = climb_result.mach[-1]
    # - fuel_consumed_climb = climb_result.cumFuel_kg[-1]
    # - current_weight = initial_mass_kg - fuel_consumed_climb

# 3. Main Simulation Execution
cruise_results = simulate_steady_cruise(
    initial_state=initial_state,
    target_distance_km=target_distance_km,
    aero=aero,
    engine=engine,
    time_step_s=time_step_s
)
```

6.2 Core Simulation Logic Flow

Function: `simulate_steady_cruise()`

Step-by-Step Execution Flow:

```
# PHASE 1: PRE-SIMULATION SETUP
def simulate_steady_cruise(initial_state, target_distance_km, aero, engine, time_step_s):

    # Step 1: Calculate True Airspeed (constant for cruise)
    true_airspeed_mps = calculate_true_airspeed(initial_state.mach, initial_state.altitude_m)
        ↓
        # Calls: calculate_atmospheric_properties(altitude_m)
        # Returns: (T, p, rho, a) where a = speed of sound
        # Formula: V_TAS = mach × a

    # Step 2: Calculate Distance per Time Step
    distance_per_step_km = (true_airspeed_mps * time_step_s) / 1000.0

    # Step 3: Estimate Total Simulation Steps
    n_steps = int(np.ceil(target_distance_km / distance_per_step_km))

    # Step 4: Initialize Trajectory Arrays
    # Arrays: time, distance, weight, fuel, thrust, drag, fuel_flow, ps, lever, altitude, mach
    # Atmospheric: temperature, density, true_airspeed
```

```
# PHASE 2: SIMULATION LOOP EXECUTION
for step in range(n_steps + 1):

    # Step 1: Get Base Drag from Aerodynamics Tables
    base_drag_N = aero.get_drag(initial_state.mach, initial_state.altitude_m)
        ↓
        # Bilinear interpolation in drag tables
        # Returns drag at current Mach and altitude

    # Step 2: Calculate Weight-Adjusted Drag
    drag_N = calculate_weight_adjusted_drag(
        base_drag_N,                    # Base drag from tables
        current_weight,                 # Current aircraft weight
        initial_state.weight_kg,        # Reference weight
        initial_state.mach,             # Mach number
        initial_state.altitude_m        # Altitude
    )
        ↓
        # Weight scaling calculation:
        # weight_ratio = current_weight / reference_weight
        # induced_drag_fraction = 0.4
        # parasitic_drag = base_drag × 0.6 (constant)
        # induced_drag = base_drag × 0.4 × weight_ratio²
        # total_drag = parasitic_drag + induced_drag

    # Step 3: Calculate Required Thrust
    thrust_required_N = calculate_required_thrust_cruise(drag_N, current_weight, altitude_m)
        ↓
        # For steady cruise: thrust = drag
        # return drag_N

    # Step 4: Find Required Lever Position
    lever_required = find_required_lever(
        engine,
        thrust_required_N,
        initial_state.mach,
        initial_state.altitude_m
    )
        ↓
```

```
        # Binary search algorithm:
        # 1. Check bounds (idle vs max thrust)
        # 2. Binary search lever range [0.0, 1.0]
        # 3. Convergence: |thrust_actual - thrust_required| < TOL/N_ENGINES
        # 4. Return optimal lever position

    # Step 5: Get Actual Engine Performance
    thrust_per_engine_N = engine.thrust_with_lever(
        lever_required,
        initial_state.mach,
        initial_state.altitude_m
    )
    thrust_total_N = N_ENGINES * thrust_per_engine_N

    # Step 6: Get TSFC and Calculate Fuel Flow
    tsfc = engine.tsfc_current()
    fuel_flow_kgps = thrust_total_N * tsfc if tsfc > 0 else 0.0

    # Step 7: Calculate Specific Excess Power
    ps = calculate_specific_excess_power(
        thrust_total_N,
        drag_N,
        current_weight,
        true_airspeed_mps
    )
        ↓
        # Formula: Ps = (T - D) × V_TAS / W
        # For steady cruise, Ps should be ≈ 0

    # Step 8: Store Current State in Arrays
    # All trajectory and atmospheric data stored for current step

    # Step 9: Check Distance Convergence
    if cumulative_distance >= target_distance_km:
        break  # End simulation

    # Step 10: Update for Next Iteration
    if step < n_steps:
        # Calculate fuel consumed this step
        fuel_step = calculate_fuel_consumption_step(thrust_total_N, tsfc, time_step_s)
```

```
            ↓
            # fuel_step = fuel_flow_kgps × time_step_s

        # Update cumulative quantities
        cumulative_fuel += fuel_step
        current_weight = update_weight_after_fuel_consumption(current_weight, fuel_step)
        cumulative_distance += distance_per_step_km
```

```
# PHASE 3: POST-SIMULATION PROCESSING
# Step 1: Calculate Summary Statistics
total_time = time_array[-1]
total_fuel = fuel_consumed_array[-1]
final_weight = weight_array[-1]
avg_fuel_flow = np.mean(fuel_flow_array[fuel_flow_array > 0])
avg_thrust = np.mean(thrust_array[thrust_array > 0])

# Step 2: Create Results Object
return CruiseResults(
    initial_state=initial_state,
    target_distance_km=target_distance_km,
    time_step_s=time_step_s,
    # ... all trajectory arrays ...
    # ... all summary statistics ...
)
```

6.3 Data Flow Through System

Input Data Flow:

```
Climb Results → CruiseInitialState → Simulation Parameters
     ↓                ↓                        ↓
Altitude/Mach    Weight/Fuel            Time Step/Distance
```

Processing Data Flow:

```
Atmospheric Properties → True Airspeed → Distance per Step
         ↓                    ↓                ↓
Temperature/Density    Velocity Calculation   Trajectory Planning
```

Engine Integration Flow:

```
Required Thrust → Lever Search → Actual Thrust → TSFC → Fuel Flow
       ↓              ↓             ↓           ↓         ↓
   Drag Balance   Binary Search   Engine Perf.  Efficiency  Consumption
```

State Update Flow:

```
Fuel Consumption → Weight Update → Drag Adjustment → Thrust Adjustment
       ↓              ↓              ↓                ↓
   TSFC × Thrust   Mass Reduction   Induced Drag    New Lever Position
```

6.4 Function Call Hierarchy

```
run_cruise_simulation()
├── extract_cruise_initial_state()
│   ├── climb_result.alt_m[-1]          # Final altitude
│   ├── climb_result.mach[-1]           # Final Mach
│   ├── climb_result.cumFuel_kg[-1]     # Climb fuel
│   └── initial_mass_kg - fuel_consumed # Current weight
│
└── simulate_steady_cruise()
    ├── calculate_true_airspeed()
    │   └── calculate_atmospheric_properties()
    │       ├── Atmosphere.calculate_atmospheric_properties()
    │       └── Atmosphere.get_speed_of_sound()
    │
    ├── [SIMULATION LOOP]
    │   ├── aero.get_drag()             # Base drag lookup
    │   ├── calculate_weight_adjusted_drag()
    │   │   └── calculate_atmospheric_properties() # For density
    │   ├── calculate_required_thrust_cruise()
    │   ├── find_required_lever()
    │   │   ├── engine.thrust_with_lever() # Binary search iterations
    │   │   └── Convergence check: |thrust_actual - required| < TOL
    │   ├── engine.thrust_with_lever()   # Final thrust
    │   ├── engine.tsfc_current()        # TSFC lookup
    │   ├── calculate_specific_excess_power()
    │   └── calculate_fuel_consumption_step()
    │
    └── [POST-PROCESSING]
        ├── np.mean() calculations       # Summary statistics
        └── CruiseResults()              # Results object creation
```

6.5 Critical Decision Points

1. Lever Position Convergence:

```
if abs(thrust_mid - required_per_engine) < THRUST_CONVERGENCE_TOL / N_ENGINES:
    return lever_mid  # Converged
```

2. Distance Target Achievement:

```
if cumulative_distance >= target_distance_km:
    break  # Mission complete
```

3. Weight Safety Check:

```
if new_weight < 0.5 * current_weight_kg:
    raise ValueError("Weight reduction too large")
```

4. Engine Performance Validation:

```
if thrust_mid is None:
    break  # Invalid engine response
```

6.6 Performance Optimization Points

1. Atmospheric Property Caching:

- Properties calculated once per altitude (constant cruise)
- Reused across all simulation steps

2. Engine Performance Optimization:

- Binary search minimizes engine model calls
- Convergence tolerance balances accuracy vs. speed

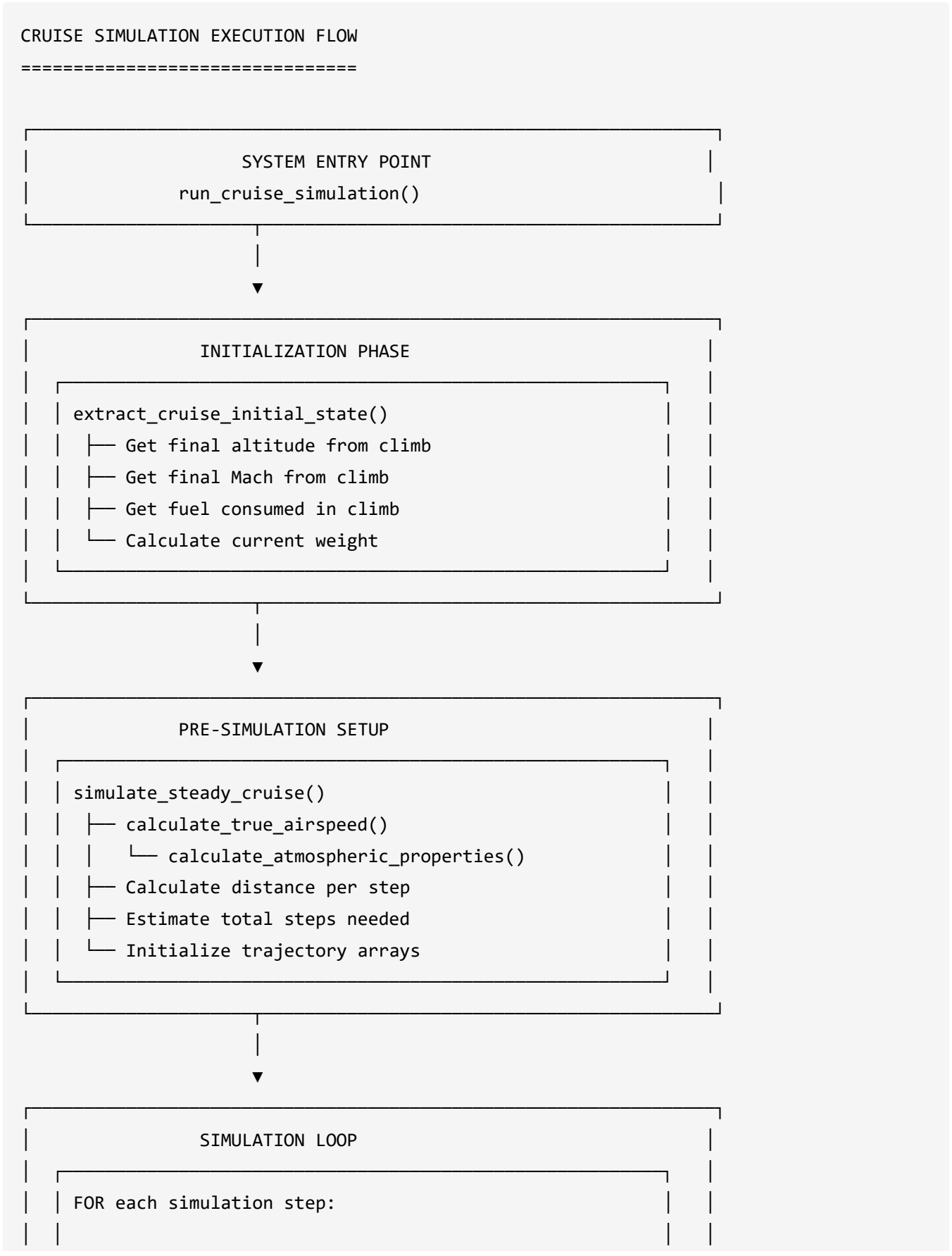
3. Array Pre-allocation:

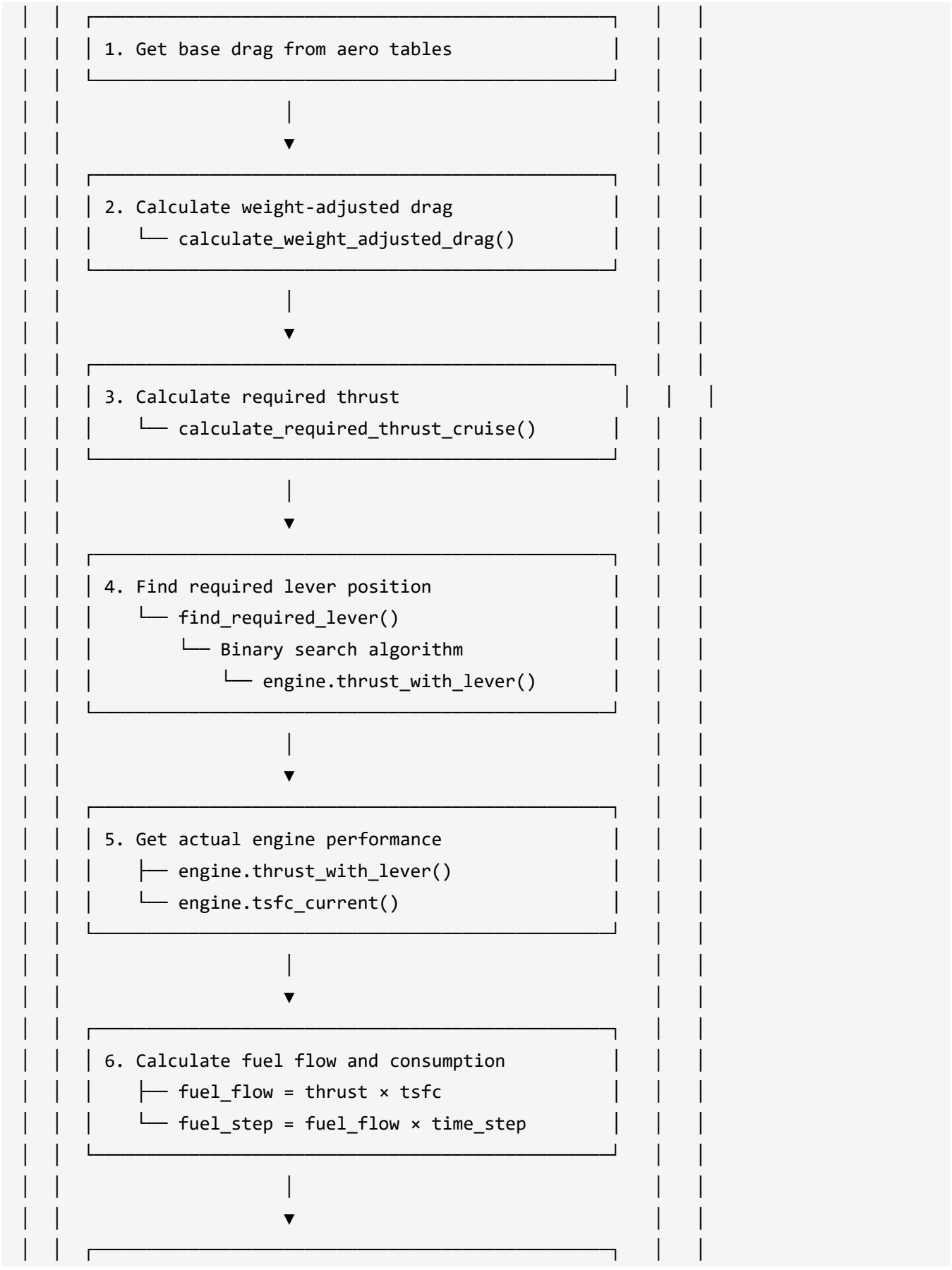
- All trajectory arrays allocated upfront
- Avoids dynamic memory allocation during simulation

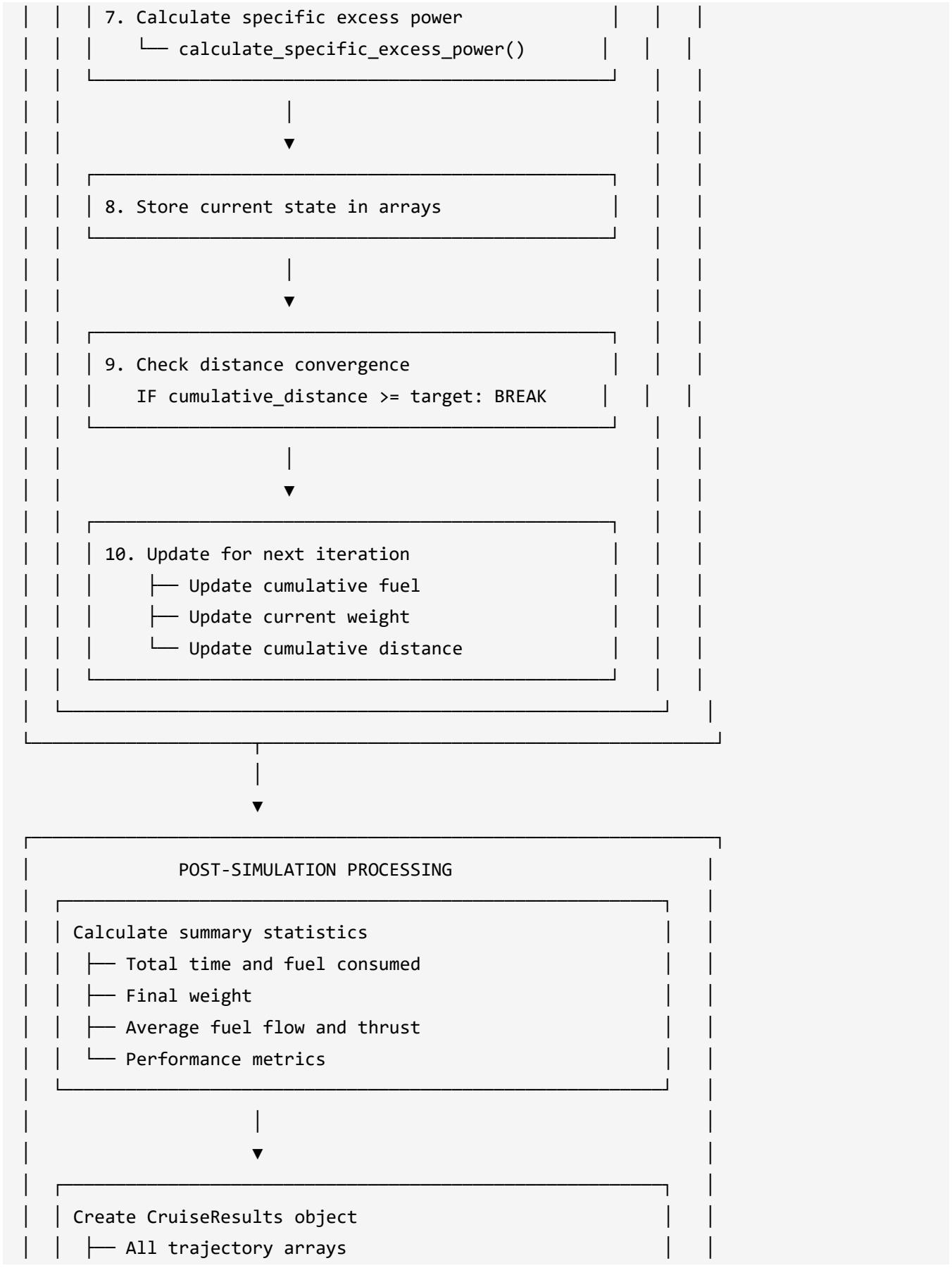
4. Early Termination:

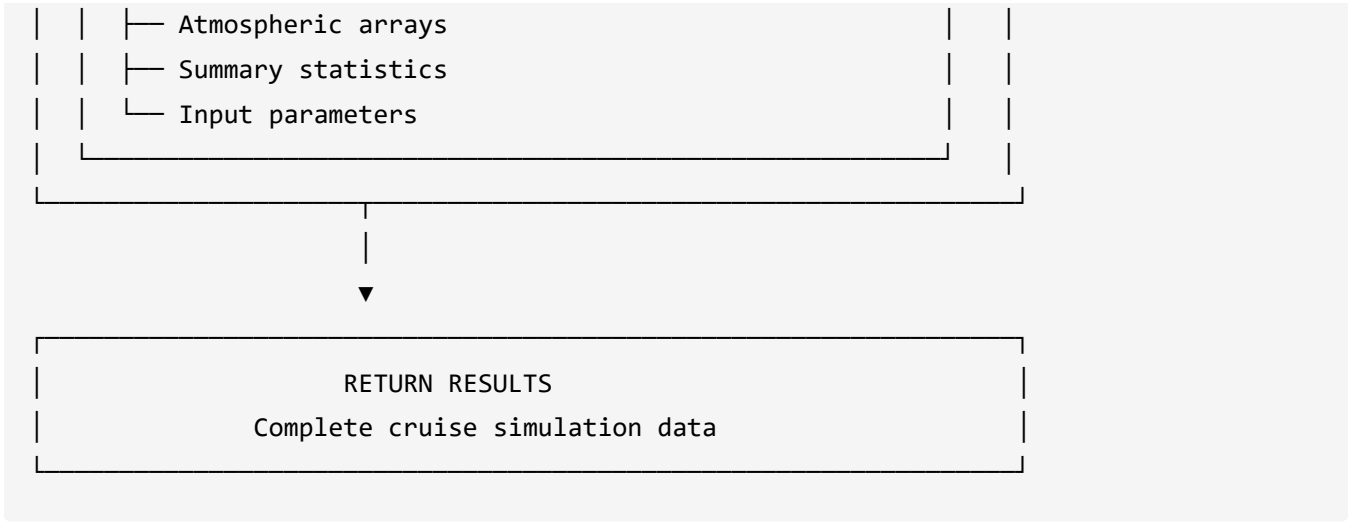
- Simulation stops when distance target reached
- Prevents unnecessary computation

6.7 Visual Code Flow Diagram


CRUISE SIMULATION EXECUTION FLOW
SYSTEM ENTRY POINT
run_cruise_simulation()
INITIALIZATION PHASE
extract_cruise_initial_state()
Get final altitude from climb
Get final Mach from climb
Get fuel consumed in climb
Calculate current weight
PRE-SIMULATION SETUP
simulate_steady_cruise()
calculate_true_airspeed()
calculate_atmospheric_properties()
Calculate distance per step
Estimate total steps needed
Initialize trajectory arrays
SIMULATION LOOP
FOR each simulation step:


1. Get base drag from aero tables
2. Calculate weight-adjusted drag
└── calculate_weight_adjusted_drag()
3. Calculate required thrust
└── calculate_required_thrust_cruise()
4. Find required lever position
└── find_required_lever()
└── Binary search algorithm
└── engine.thrust_with_lever()
5. Get actual engine performance
├── engine.thrust_with_lever()
└── engine.tsfc_current()
6. Calculate fuel flow and consumption
├── fuel_flow = thrust × tsfc
└── fuel_step = fuel_flow × time_step


7. Calculate specific excess power
calculate_specific_excess_power()
8. Store current state in arrays
9. Check distance convergence
IF cumulative_distance >= target: BREAK
10. Update for next iteration
Update cumulative fuel
Update current weight
Update cumulative distance
POST-SIMULATION PROCESSING
Calculate summary statistics
Total time and fuel consumed
Final weight
Average fuel flow and thrust
Performance metrics
Create CruiseResults object
All trajectory arrays



6.8 Key Data Transformations

Weight Evolution:

```
Initial Weight → Fuel Consumption → Weight Reduction → Drag Adjustment
     ↓               ↓                    ↓                ↓
65,000 kg       TSFC × Thrust × Δt    Mass Reduction   Induced Drag ↓
```

Thrust-Drag Balance:

```
Drag Calculation → Required Thrust → Lever Position → Actual Thrust
       ↓                ↓                ↓               ↓
   Parasitic +      Drag = Thrust   Binary Search   Engine Response
   Induced Drag                        Algorithm
```

Fuel Flow Chain:

```
Lever Position → Engine Thrust → TSFC Lookup → Fuel Flow → Consumption
       ↓              ↓             ↓           ↓           ↓
   0.0 - 1.0      Thrust(N)    kg/(N·s)    kg/s        kg/step
```

---

7) Simulation Engine Implementation

6.1 Main Simulation Function

Function:

`simulate_steady_cruise(initial_state, target_distance_km, aero, engine, time_step_s)`

Purpose: Simulate complete steady level cruise trajectory with dynamic weight and thrust adjustments.

Simulation Flow:

```
def simulate_steady_cruise(initial_state: CruiseInitialState,
                           target_distance_km: float,
                           aero: AeroTables,
                           engine: EngineWrapper,
                           time_step_s: float = DEFAULT_TIME_STEP_S) -> CruiseResults:
    """Simulate steady level cruise at constant Mach and altitude."""

    # Calculate true airspeed (constant for constant Mach and altitude)
    true_airspeed_mps = calculate_true_airspeed(initial_state.mach, initial_state.altitude_m)
    distance_per_step_km = (true_airspeed_mps * time_step_s) / 1000.0

    # Estimate number of steps needed
    n_steps = int(np.ceil(target_distance_km / distance_per_step_km))

    # Initialize trajectory arrays
    # ... array initialization ...

    # Simulation loop
    for step in range(n_steps + 1):
        # Get base drag and adjust for current weight
        base_drag_N = aero.get_drag(initial_state.mach, initial_state.altitude_m)
        drag_N = calculate_weight_adjusted_drag(
            base_drag_N, current_weight, initial_state.weight_kg,
            initial_state.mach, initial_state.altitude_m
        )

        # Calculate required thrust
        thrust_required_N = calculate_required_thrust_cruise(drag_N, current_weight,
                                                             initial_state.altitude_m)

        # Find required lever position
        lever_required = find_required_lever(engine, thrust_required_N,
                                             initial_state.mach, initial_state.altitude_m)

        # Get actual thrust and TSFC
        thrust_per_engine_N = engine.thrust_with_lever(lever_required, initial_state.mach,
                                                       initial_state.altitude_m)
        thrust_total_N = N_ENGINES * thrust_per_engine_N if thrust_per_engine_N else 0.0
        tsfc = engine.tsfc_current()
```

```
        # Calculate fuel flow and consumption
        fuel_flow_kgps = thrust_total_N * tsfc if tsfc > 0 else 0.0

        # Store current state
        # ... store arrays ...

        # Update for next step
        if step < n_steps:
            fuel_step = calculate_fuel_consumption_step(thrust_total_N, tsfc, time_step_s)
            cumulative_fuel += fuel_step
            current_weight = update_weight_after_fuel_consumption(current_weight, fuel_step)
            cumulative_distance += distance_per_step_km

    return CruiseResults(...)
```

6.2 Simulation Parameters

Core Parameters:

```
# Default cruise parameters
DEFAULT_TIME_STEP_S = 60.0    # 1 minute time steps
DEFAULT_DISTANCE_KM = 1000.0  # Default cruise distance
GRAVITY_MS2 = 9.80665         # Standard gravity

# Convergence criteria
THRUST_CONVERGENCE_TOL = 1.0   # Newton tolerance for thrust balance
MAX_ITERATIONS = 50            # Maximum iterations for convergence

# Safety limits
MIN_CRUISE_MACH = 0.3         # Minimum safe cruise Mach
MAX_CRUISE_MACH = 0.9         # Maximum reasonable cruise Mach
MIN_CRUISE_ALT_M = 1000.0     # Minimum cruise altitude
MAX_CRUISE_ALT_M = 15000.0    # Maximum cruise altitude
```

Parameter Effects:

- **DEFAULT_TIME_STEP_S**: Controls simulation resolution and computational cost
- **THRUST_CONVERGENCE_TOL**: Affects lever positioning accuracy

- **Safety limits**: Ensure realistic operating conditions

---

7) Mission Integration and Interface

7.1 Climb Phase Integration

Function: `extract_cruise_initial_state(climb_result, initial_mass_kg)`

Purpose: Extract initial cruise state from climb optimization results.

Integration Process:

```
def extract_cruise_initial_state(climb_result: MinFuelSchedule,
                                 initial_mass_kg: float) -> CruiseInitialState:
    """Extract initial cruise state from climb optimization results."""

    # Get final state from climb
    final_altitude = float(climb_result.alt_m[-1])
    final_mach = float(climb_result.mach[-1])
    fuel_consumed_climb = float(climb_result.cumFuel_kg[-1])
    climb_time = float(np.sum(climb_result.dt_s))

    # Calculate current weight after climb
    current_weight = initial_mass_kg - fuel_consumed_climb

    return CruiseInitialState(
        altitude_m=final_altitude,
        mach=final_mach,
        weight_kg=current_weight,
        fuel_consumed_climb_kg=fuel_consumed_climb,
        climb_time_s=climb_time
    )
```

Key Features:

- Seamless transition from climb to cruise
- Weight accounting for climb fuel consumption

- Validation of cruise initial conditions

7.2 Main Interface Function

Function:

`run_cruise_simulation(climb_result, initial_mass_kg, target_distance_km, aero, engine, time_step_s,`

Purpose: Main interface for complete cruise simulation with mission integration.

Interface Flow:

```
def run_cruise_simulation(climb_result: MinFuelSchedule,
                          initial_mass_kg: float,
                          target_distance_km: float,
                          aero: AeroTables,
                          engine: EngineWrapper,
                          time_step_s: float = DEFAULT_TIME_STEP_S,
                          create_plots: bool = True) -> CruiseResults:
    """Main function to run complete cruise simulation."""

    # Extract initial state from climb
    initial_state = extract_cruise_initial_state(climb_result, initial_mass_kg)

    # Run cruise simulation
    cruise_results = simulate_steady_cruise(
        initial_state=initial_state,
        target_distance_km=target_distance_km,
        aero=aero,
        engine=engine,
        time_step_s=time_step_s
    )

    return cruise_results
```

7.3 Mission Analysis Benefits

Seamless Integration: Direct connection between climb optimization and cruise simulation
Weight Continuity: Accurate weight tracking across mission phases
Performance Validation: Realistic cruise performance based on climb results

Fuel Accounting: Complete fuel consumption tracking for mission analysis

---

8) Validation and Quality Assurance

8.1 Input Validation

CruiseInitialState Validation:

- Altitude range checking (1000-15000m)
- Mach number validation (0.3-0.9)
- Positive weight verification
- Automatic validation in dataclass `__post_init__`

Weight Safety Checks:

```
def update_weight_after_fuel_consumption(current_weight_kg: float,
                                         fuel_consumed_kg: float) -> float:
    """Update aircraft weight after fuel consumption."""
    new_weight = current_weight_kg - fuel_consumed_kg

    # Ensure weight doesn't become unreasonably low
    if new_weight < 0.5 * current_weight_kg:
        raise ValueError(f"Weight reduction too large: {current_weight_kg:.1f} -> {new_weight:.1f}

    return new_weight
```

8.2 Performance Monitoring

Convergence Monitoring:

- Thrust balance convergence tracking
- Lever position optimization monitoring
- TSFC validation and error handling

Output Validation:

- Array length consistency checking
- Physical quantity validation (positive values)
- Summary statistics verification

8.3 Error Handling

Engine Performance Errors:

- Invalid thrust calculations
- TSFC retrieval failures
- Lever position convergence issues

Simulation Errors:

- Weight reduction validation
- Distance calculation errors
- Array allocation failures

End of Cruise Phase Documentation